LungLogic Dual-Branch Lung Model Control Software Stage Report

Yuqiao Li and the LungLogic Project Team

August 21, 2026

Abstract

This report documents the current control-layer implementation of the LungLogic prototype. It focuses on how user-facing requirements, including respiratory rate, tidal volume, inspiration-to-expiration ratio, target flow rate, and left/right asymmetry, are converted into software states, target flows, valve commands, and real-time monitoring data. Mechanical analysis is treated only as background for model parameter selection. The present software uses a first-order grey-box model as the basis for the digital twin and feedforward control. If future bench data shows significant overshoot, oscillation, delay, or inertial behaviour, the model can be upgraded to a second-order or delayed model.

Contents

1 Stage Objectives and Control Boundary **3**

2 Mapping User Requirements to Control Parameters **3**

3 Control Logic and Mathematical Formulation **5**

3.1 From User Settings to Breathing Timing 5

3.2 Reference Flow Generation 6

3.3 First-Order Lung Branch Model 6

3.4 Model-Based Feedforward 6

3.5 PID Feedback Correction 7

3.6 Valve Command Conversion 7

3.7 Volume-Based Termination and Independent Branch Control 7

3.8 Model Parameters and Calibration Status 8

3.9 Conditions for Upgrading to a Second-Order Model 8

4 Control State Machine **8**

5 Control Software Implementation **9**

5.1 Simulation Implementation: `lung_pid_gui.py` 9

5.2 Hardware Monitor and Serial Protocol: `lung_hardware_monitor.py` and Arduino Firmware 10

6 Hardware Migration and Calibration Plan **11**

6.1 From Simulation Parameters to Hardware Commands 12

6.2 Required Calibrations 12

6.3 Role of `lung_hardware_monitor.py` 13

6.4 Future Calibration Interface 13

7 Compliance with Scope **13**

8 Current Limitations and Next Steps **14**

9 Conclusion **15**

1 Stage Objectives and Control Boundary

The objective of this stage is to establish a runnable, tunable, and loggable dual-branch lung control software framework. The software is intended to support both simulation-stage design and later hardware closed-loop testing.

The current system boundary is:

- No positive pressure source is used. During inspiration, the lung phantom expands passively by material rebound while connected to atmosphere. During expiration, a regulated vacuum source actively contracts the lung phantom.
- The left and right lung branches are controlled independently. Each side contains an atmosphere routing valve, a vacuum routing valve, a PPV40V proportional pinch valve, and an SFM3300 bidirectional flow sensor.
- The control layer uses a 100 Hz control cycle for the breathing state machine, flow PID control, volume estimation, valve interlocking, fault handling, and telemetry.
- The Python host software is responsible for configuration, simulation tuning, visualisation, serial communication, data logging, and post-experiment plotting.

At this stage, the output should be understood as a control and software prototype, not a final hardware validation result. Stability, leakage, pressure range, and long-term repeatability still require bench testing.

2 Mapping User Requirements to Control Parameters

The project requirements include adjustable respiratory rate, tidal volume, I:E ratio, inspiration/expiration flow rates, and controllable left/right imbalance. The software maps these requirements to the variables shown in Table 1.

Table 1: Mapping from user settings to control variables

User setting	**Software variable**	**Control meaning**
Respiratory rate	`respiratory_rate_bpm`	Defines the breathing period $T_c = 60/RR$. The nominal value is 15 bpm, corresponding to 4 s/cycle.
I:E ratio	`ie_inspiration`, `ie_expiration`	Splits the cycle into inspiration and expiration times: $T_{in} = T_c I/(I+E)$ and $T_{out} = T_c E/(I+E)$. The nominal ratio is 1:2.
Tidal volume	`left_target_ml`, `right_target_ml`	Independent target volume for each lung branch. The nominal setting is 250 mL per side, or 500 mL total.
Target fill time	`target_fill_time_s`	Converts the volume target into a target inspiration flow rate. The nominal value is 1.3 s.
Ventilation imbalance	Left/right targets and model parameters	Allows simulated left/right imbalance through different volume targets, time constants, or flow signs.
Direction-change dead time	`dead_time_s`	Closes all routing valves during direction changes to avoid connecting atmosphere and vacuum at the same time.
Precondition time	`precondition_s`	Applies vacuum before the first cycle to establish a repeatable collapsed initial condition.

Table 2 summarises the main mathematical symbols used in the report.

Table 2: Notation used in the control report

Symbol	Meaning	Description
RR	Respiratory rate	Breaths per minute, used to compute the breathing cycle period.
T_c	Cycle period	$T_c = 60/RR$, in seconds.
I, E	I:E ratio terms	Inspiration and expiration ratio terms used to split the cycle.
T_{in}	Inspiration time	$T_{in} = T_c I/(I+E)$.
T_{out}	Expiration time	$T_{out} = T_c E/(I+E)$.
T_{fill}	Target fill time	Time used to convert tidal volume target into reference inspiratory flow.
T_{dead}	Dead time	All-routing-valves-closed interval during direction changes.
i	Branch index	Refers to the left or right lung branch.
V_i	Estimated branch volume	Integrated branch volume, in mL.
$V^*_{T,i}$	Target tidal volume	Branch-specific inspiration termination target, in mL.
$V_{\max,i}$	Maximum model volume	Capacity limit used by the first-order branch model.
V_{end}	End-expiration volume	Volume threshold for stopping vacuum contraction.
Q_i	Measured branch flow	SFM3300 flow in SLM; positive in inspiration and negative in expiration.
Q^*_i	Reference flow	Target flow magnitude tracked by the PID controller.
$Q^*_{in,i}$	Reference inspiratory flow	Computed from $Q^*_{in,i} = 0.06V^*_{T,i}/T_{\text{fill}}$.
$Q^*_{out,i}$	Reference expiratory flow	Target flow magnitude during vacuum contraction.
$Q_{\max,i}$	Full-open available flow	Estimated from current volume, capacity, and time constant for feedforward.
$\dot{V}_i$	Volume rate	Related to flow by $Q_i = 0.06\dot{V}_i$.
Δt	Control time step	At 100 Hz, $\Delta t = 0.01$ s.
a_i	Normalised PPV40V opening	$a_i = 0$ closed, $a_i = 1$ fully open.
$a_{ff,i}$	Feedforward opening	Base PPV40V opening estimated from the first-order model.
U_i	PPV40V command voltage	$U_i = 5(1-a_i)$; 0 V fully open and 5 V fully closed.
γ	Valve exponent	Nonlinear exponent between PPV40V opening and effective flow.
$\tau_{in,i}$	Inspiration time constant	Expansion speed when branch i is connected to atmosphere.
$\tau_{out,i}$	Expiration time constant	Contraction speed when branch i is connected to vacuum.
e_i	Flow error	$e_i = Q^*_i - \lvert Q_i \rvert$ for PID control.
d_i	Derivative input	$d_i = -d\lvert Q_i \rvert/dt$, used as a damping term.
K_p, K_i, K_d	PID gains	Proportional, integral, and derivative gains.
L	Explicit delay	Total delay term in a possible delayed model.
ω_n, ζ	Second-order parameters	Natural frequency and damping ratio for a possible second-order model.

3 Control Logic and Mathematical Formulation

This section reorganises the control logic as a step-by-step path from user settings to valve commands. The sequence is: breathing timing, reference flow generation, first-order lung modelling, model-based feedforward, PID feedback correction, actuator voltage conversion, and finally volume-based branch termination.

3.1 From User Settings to Breathing Timing

The respiratory rate defines the duration of one complete breathing cycle:

$$T_c = \frac{60}{RR}. \tag{1}$$

The I:E ratio then divides this cycle into inspiration and expiration:

$$T_{in} = T_c \frac{I}{I+E}, \qquad T_{out} = T_c \frac{E}{I+E}. \tag{2}$$

For the nominal setting of 15 bpm and I:E = 1:2, the cycle time is 4 s, giving approximately 1.33 s for inspiration and 2.67 s for expiration.

3.2 Reference Flow Generation

The target inspiratory flow is computed from the desired tidal volume and target fill time:

$$Q^*_{in,i} = 0.06 \frac{V^*_{T,i}}{T_{\text{fill}}}. \tag{3}$$

The factor 0.06 converts mL/s to SLM. With the nominal setting of 250 mL per side and 1.3 s fill time:

$$Q^*_{in} = 0.06 \times \frac{250}{1.3} \approx 11.54 \text{ SLM}. \tag{4}$$

During expiration, the measured flow is negative because the vacuum source contracts the lung phantom. The controller therefore tracks the magnitude of the flow rather than its signed value. In both inspiration and expiration, the feedback error is written as:

$$e_i = Q^*_i - |Q_i|. \tag{5}$$

3.3 First-Order Lung Branch Model

Each lung branch is modelled as a first-order variable-volume system:

$$\dot{V}_i = \begin{cases} a_i^{\gamma}(V_{\max,i} - V_i)/\tau_{in,i}, & \text{inspiration}, \\ -a_i^{\gamma} V_i/\tau_{out,i}, & \text{expiration}, \\ 0, & \text{isolated}. \end{cases} \tag{6}$$

The flow-volume relationship is:

$$Q_i = 0.06\dot{V}_i, \qquad V_i[k] = V_i[k-1] + \frac{1000}{60} Q_i[k]\Delta t. \tag{7}$$

Here V_i is the estimated branch volume, $V_{\max,i}$ is the maximum model volume, $\tau_{in,i}$ describes passive rebound when the branch is connected to atmosphere, and $\tau_{out,i}$ describes contraction under vacuum. The opening a_i is the normalised PPV40V opening, and γ captures the nonlinear relation between valve opening and effective flow.

3.4 Model-Based Feedforward

The first-order model is used to estimate a base valve opening before feedback correction is applied. This reduces the amount of work left to the PID controller:

$$a_{ff,i} = \left(\frac{Q^*_i}{Q_{\max,i}} \right)^{1/\gamma}. \tag{8}$$

During inspiration, the full-open available flow is estimated as:

$$Q_{\max,i} = 0.06 \frac{V_{\max,i} - V_i}{\tau_{in,i}}. \tag{9}$$

During expiration, it is estimated as:

$$Q_{\max,i} = 0.06 \frac{V_i}{\tau_{out,i}}. \tag{10}$$

3.5 PID Feedback Correction

The PID controller corrects the model-based opening using measured flow feedback:

$$a_i = \mathrm{sat}_{[0,1]} \left[a_{ff,i} + K_p e_i + K_i \int e_i dt + K_d d_i \right]. \tag{11}$$

The derivative term uses the measurement derivative:

$$d_i = -\frac{d|Q_i|}{dt}. \tag{12}$$

This avoids differentiating the reference step directly and reduces derivative kick during state transitions. The default value is $K_d = 0$, so the controller behaves like the previous PI implementation until derivative damping is tuned on hardware.

The current default controller parameters are:

- $K_p = 0.022$ opening/SLM;
- $K_i = 0.16$ opening/(SLM $\cdot$ s);
- $K_d = 0.0$ opening $\cdot$ s/SLM;
- integral limit: 2.0;
- conditional integration for anti-windup.

3.6 Valve Command Conversion

The PID output is an opening command, but the PPV40V receives a voltage command. The conversion used by both the simulation and hardware firmware is:

$$U_i = 5(1 - a_i). \tag{13}$$

With the current project convention, $a_i = 1$ corresponds to 0 V and a fully open PPV40V, while $a_i = 0$ corresponds to 5 V and a fully closed PPV40V. On the hardware, Arduino PWM outputs are filtered by RC low-pass circuits to generate this approximate 0–5 V command.

3.7 Volume-Based Termination and Independent Branch Control

Flow control regulates how quickly each branch changes volume, but branch termination is based on estimated volume. During inspiration, a branch is closed when:

$$V_i \geq V_{T,i}^*. \tag{14}$$

During expiration, vacuum contraction stops when:

$$V_i \leq V_{\mathrm{end}}. \tag{15}$$

The left and right branches are evaluated independently. If one side reaches its target first, that side can close while the other continues. This is the basis for asymmetric ventilation simulation, where imbalance can be introduced through different left/right tidal-volume targets, time constants, PPV40V mappings, sensor signs, or zero offsets.

3.8 Model Parameters and Calibration Status

The initial model parameters were selected from the mechanical team's lung phantom and tubing report. The report approximated rebound as:

$$V(t) = V_{\max}(1 - e^{-t/\tau}). \tag{16}$$

The reported geometry gave a maximum displaced volume of approximately 558 mL. For 10 mm inner diameter tubing, the reported recovery time was approximately $4\tau = 2.27$ s, giving:

$$\tau_{in} \approx 2.27/4 \approx 0.57 \text{ s}. \tag{17}$$

The current initial parameter set is:

- left and right $V_{\max} = 558$ mL;
- left $\tau_{in} = 0.57$ s and right $\tau_{in} = 0.60$ s;
- left/right $\tau_{out} = 0.50/0.54$ s;
- valve exponent $\gamma = 1.15$;
- transport delay = 25 ms;
- sensor time constant = 20 ms;
- flow noise = 0.10 SLM.

These values are initial simulation assumptions rather than final hardware calibration results. At the hardware stage, PPV40V voltage-flow tests should be used to update γ or replace it with a lookup table, valve-step tests should estimate transport delay, SFM3300 step responses should identify sensor dynamics, and steady zero-flow/steady-flow recordings should estimate sensor noise.

3.9 Conditions for Upgrading to a Second-Order Model

The first-order model is suitable for early control design because it is simple and easy to identify from bench data. A second-order or delayed model should be considered if hardware tests show clear underdamped flow oscillation, a rising edge that cannot be fitted by a first-order response, significant PPV40V mechanical lag, or strong coupling between the two branches.

A possible second-order form is:

$$G(s) = \frac{\omega_n^2}{s^2 + 2\zeta\omega_n s + \omega_n^2} e^{-Ls}, \tag{18}$$

where ω_n is the natural frequency, ζ is the damping ratio, and L is an explicit delay.

4 Control State Machine

The breathing cycle is divided into multiple states rather than only inspiration and expiration. This ensures valve interlocking, avoids direct atmosphere-vacuum cross-flow, and establishes a repeatable initial condition.

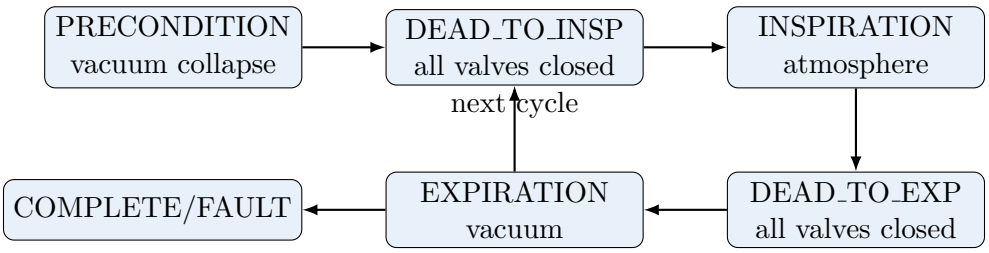


Figure 1: Breathing state machine used by the current control software

Table 3: Valve behaviour in each state

State	**Atmosphere valve**	**Vacuum valve**	**Control objective**
PRECONDITION	closed	open	Collapse the lung phantom to a repeatable initial condition.
DEAD_TO_INSP	closed	closed	Direction-change dead time.
INSPIRATION	open	closed	Regulate positive flow until the target tidal volume is reached.
DEAD_TO_EXP	closed	closed	Direction-change dead time before vacuum contraction.
EXPIRATION	closed	open	Regulate negative flow until end-expiration volume is reached.
COMPLETE/FAULT	closed	closed	Stop the run or enter a safe state.

At no point should the atmosphere and vacuum valves on the same branch be open simultaneously. This is checked in both the simulation logic and the Arduino firmware.

5 Control Software Implementation

5.1 Simulation Implementation: `lung_pid_gui.py`

`lung_pid_gui.py` is the simulation-stage GUI. It does not directly control Arduino or physical valves. Instead, it calls the first-order model and state machine in `lung_model.py` to generate a repeatable digital-twin result. Its purpose is to verify the control intent, parameter ranges, and state transitions before hardware testing.

The simulation GUI currently provides:

- adjustment of respiratory rate, I:E ratio, left/right tidal volume targets, target fill time, and simulation cycles;
- adjustment of PID gains, integral limit, and model feedforward;
- adjustment of branch capacity, inspiration and expiration time constants, valve exponent, transport delay, sensor noise, and dead time;
- a dynamic pneumatic diagram showing atmosphere, vacuum, routing valves, PPV40V, SFM3300, and both lung branches;
- time histories of volume, flow, PPV opening, and state;
- a live monitor window showing flow rate, reference flow, volume, PPV opening, command voltage, routing valves, and live flow curves.

The word "live" in the simulation GUI means live playback of precomputed simulation data. It is not hardware telemetry from SFM3300 sensors.

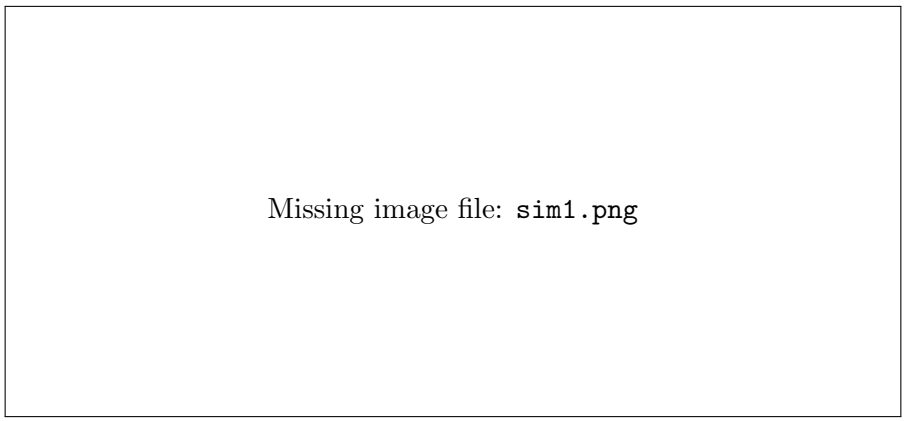


Figure 2: Current simulation GUI showing the parameter panel, Dynamic Model view, and Live Curves tab



Figure 3: Simulation-stage live monitor showing state, cycle number, measured and reference flow, volume, PPV command, routing valves, and rolling flow curve

In addition, `lung_pid_simulation.py` provides a command-line simulation path that writes `output/lung_simula` and `output/lung_simulation.png`. This allows the same model to support both GUI demonstration and batch simulation.

5.2 Hardware Monitor and Serial Protocol: `lung_hardware_monitor.py` and Arduino Firmware

`lung_hardware_monitor.py` is the hardware-facing host program. It sends the same configuration parameters to Arduino Uno, starts the physical control loop, receives SFM3300/controller telemetry, saves CSV logs, and produces result plots. It is the main software path from simulation to hardware testing.

The serial workflow is:

1. The host opens the serial port and sends `HELLO`.
2. Arduino replies with `READY,LUNGLOGIC_UNO,2`.
3. The host sends `CFG,2`, including breathing parameters, PID gains, volume limits, model parameters, and valve exponent.

4. After acknowledgement, the host sends `START`.
5. During operation, the host sends `PING` as a keep-alive signal, while Arduino sends `TEL` telemetry at 50 Hz.
6. The host saves CSV data and plots volume, flow, and PPV opening.

Telemetry includes state, cycle index, measured left/right flow, estimated left/right volume, reference flow, PPV opening, command voltage, and valve mask.

The Arduino firmware `firmware/lung_controller_uno/lung_controller_uno.ino` currently implements:

- 100 Hz control loop;
- TCA9548A/PCA9548A Pa.Hub channel selection for the two SFM3300 sensors;
- D5/D6 PWM outputs filtered by RC low-pass circuits to generate two 0–5 V PPV40V commands;
- D2 active-low emergency stop input;
- four interlocked solenoid valve outputs;
- the same state machine and PID structure as the Python model;
- fault checks for sensor read failure, flow limit, volume limit, host timeout, and valve interlock;
- safe fault action: close all routing valves and command the PPV40V valves to approximately 5 V.

According to the current PCB report, the Arduino Uno pin allocation is:

Table 4: Arduino Uno pin mapping used by the current hardware firmware

Pin	**Function**	**Type**
D2	Emergency stop input	digital input, active-low
D3	Right ATM solenoid valve	digital output
D4	Left ATM solenoid valve	digital output
D5	Right PPV40V command	PWM output through RC filter
D6	Left PPV40V command	PWM output through RC filter
D7	Right VAC solenoid valve	digital output
D8	Left VAC solenoid valve	digital output
A4	I2C SDA	communication
A5	I2C SCL	communication

The 24 V solenoid valves and PPV40V power cannot be driven directly from Arduino pins. They require suitable MOSFET stages, flyback protection, external power, and a shared ground. Software safety checks do not replace a mechanical vacuum regulator or a hardware emergency stop.

6 Hardware Migration and Calibration Plan

When moving from simulation to hardware, the control logic can remain mostly the same, but the model parameters, sensor offsets, valve mapping, and safety limits must be calibrated experimentally. The current codebase provides three main entry points:

- `config.example.json`: shared configuration for simulation and hardware;
- `lung_hardware_monitor.py`: sends configuration to Arduino and records telemetry;
- `fit_lung_parameters.py`: fits first-order time constants from CSV data.

6.1 From Simulation Parameters to Hardware Commands

In hardware mode, the user settings are translated in the same way:

- respiratory rate and I:E ratio become inspiration, expiration, and cycle times;
- left/right tidal volume targets become branch-specific volume termination conditions;
- target fill time becomes target inspiration flow;
- dead time becomes the all-valves-closed interval between inspiration and expiration;
- PID and model parameters become PPV40V PWM duty cycles, which are filtered into 0–5 V command voltages.

The inspiration/expiration switching gap is manually adjustable through `dead_time_s`. In the GUI it appears as `Direction-change dead time / s`; in Arduino it is sent as `deadTimeMs`.

6.2 Required Calibrations

1. SFM3300 zero-flow offset calibration. When all routing valves are closed, the true flow should be 0 SLM. Real sensors may still report a small offset. If this offset is integrated, the estimated volume will drift. A recommended calibration is to average the measured flow during each dead-time interval and subtract it:

$$Q_{\text{corrected},i} = Q_{\text{measured},i} - Q_{\text{offset},i}. \tag{19}$$

2. Atmospheric rebound time constant τ_{in}. This parameter describes how fast the fake lung re-expands after being compressed or vacuum-collapsed and then connected to atmosphere. The current value of approximately 0.57 s comes from the mechanical team's 10 mm tubing test. The test should be repeated with the final hardware by fully opening the PPV40V, opening the atmosphere valve, recording SFM3300 flow, integrating volume, and fitting the first-order model. The 1.3 s target fill time is normally kept fixed unless the lung material rebound degrades significantly.

3. Vacuum contraction time constant τ_{out}. This parameter describes how fast the vacuum source contracts the lung phantom. It is currently an initial placeholder and must be updated using controlled vacuum tests.

4. PPV40V voltage-flow calibration. The PPV40V command voltage and actual flow are not perfectly linear. The current model approximates:

$$Q \propto a^{\gamma}, \qquad U = 5(1 - a). \tag{20}$$

Hardware step tests should scan 0–5 V commands in both inspiration and expiration directions. If the exponent model is sufficient, update `valve_exponent`. If there is strong nonlinearity or a dead zone, a lookup table or piecewise map should be used.

5. Sensor delay, noise, and direction calibration. The signs, noise levels, and response delays of the two SFM3300 sensors must be verified. The current delay and noise values are initial assumptions.

6. Left/right branch consistency calibration. The two branches should be calibrated independently. The final system should have separate left/right `tau_in`, `tau_out`, flow sign, zero offset, and PPV mapping values.

6.3 Role of `lung_hardware_monitor.py`

`lung_hardware_monitor.py` has three roles in hardware migration.

First, it is a configuration tool. It reads `config.example.json` and sends the full `CFG` command to Arduino.

Second, it is a run-control tool. It sends `START`, `STOP`, and periodic `PING` messages.

Third, it is the source of calibration data. It records `TEL` telemetry to CSV, including measured flow, estimated volume, reference flow, opening, command voltage, state, and valve mask. These CSV files can be used by `fit_lung_parameters.py` and later cycle-analysis tools.

6.4 Future Calibration Interface

The current hardware calibration workflow is script- and CSV-based. Future GUI support may include:

- manual dead-time setting and testing;
- PPV40V fully-open recovery tests and automatic time-to-250-mL calculation;
- vacuum contraction tests and automatic `tau_out` fitting;
- PPV40V 0–5 V step scanning;
- per-cycle zero-flow offset estimation;
- automatic tidal volume error, cycle timing error, left/right balance error, and $\pm 3\%$ stability reporting.

7 Compliance with Scope

Table 5 summarises the current control-layer status relative to the project scope.

Table 5: Current control-layer compliance with scope

Requirement	**Status**	**Comment**
12–18 bpm, nominal 15 bpm	Implemented	Supported by GUI and configuration file.
500 mL nominal tidal volume	Implemented in simulation	Default is 250 mL per side; hardware validation is still required.
Adjustable I:E ratio, nominal 1:2	Implemented	RR and I:E determine inspiration and expiration times.
Independent left/right control	Implemented in control logic	Separate targets, PID control, termination, and valve commands.
Controllable ventilation imbalance	Implemented in software	Achieved through left/right target and model parameter differences.
Real-time adjustment and visualisation	Implemented on host side	GUI supports parameter setting, dynamic model, curves, and live monitor.
Data acquisition and analysis	Partially implemented	CSV logging and plotting exist; automatic abnormality metrics still need work.
Cycle-to-cycle recalibration	Partially implemented	Preconditioning and controller reset exist; automatic zero-flow offset estimation is not complete.
$\pm 3\%$ stability	To be verified	Requires repeated hardware cycle statistics.
Pressure range and leakage proof	To be verified	Current software has no pressure sensor input or leakage test result.

8 Current Limitations and Next Steps

The current software expresses the intended control behaviour, but several limitations remain:

- τ_{out}, PPV40V nonlinearity, and sensor dynamics are still initial values;
- automatic per-cycle zero-flow offset estimation has not yet been added;
- the simulation GUI is not equivalent to hardware telemetry;
- an automatic compliance report is not yet implemented;
- without pressure sensor input, software alone cannot prove pressure safety.

Recommended next steps are:

1. use SFM3300 data to re-identify τ_{in} and τ_{out};
2. perform PPV40V 0–5 V step tests to fit voltage-flow mapping;
3. add dead-time zero-flow offset estimation to Arduino firmware;
4. add Python cycle analysis for RR, I:E, tidal volume error, left/right balance, and stability;
5. connect the GUI live monitor to real hardware telemetry;
6. upgrade to a second-order or delayed model if first-order behaviour is insufficient.

9 Conclusion

The current LungLogic control layer provides a complete software path from user requirements to control actions. Respiratory rate, tidal volume, and I:E ratio are converted into cycle timing and reference flows. A state machine safely switches between preconditioning, dead time, inspiration, and expiration. A feedforward plus PID controller converts the target flow into PPV40V opening, and volume-based termination supports symmetric or asymmetric ventilation.

The host-side GUI supports parameter adjustment, digital-twin visualisation, real-time simulated flow monitoring, and curve display. The hardware monitor and Arduino firmware provide serial configuration, telemetry logging, PWM-based PPV40V command output, emergency stop input, and basic fault protection.

The main value of this stage is the implementation of the control logic and software workflow. Final performance claims still require hardware data from the SFM3300 sensors, updated model parameters, and cycle-level analysis tied directly to the project scope.

References

[1] Mechanical design team, *System Dynamics and Volume Constraints / Orifice Report*, 2026.

[2] Electrical design team, *LungLogic Week 4 Progress Report: Electrical Section*, 2026.

[3] LungLogic codebase: `lung_model.py`, `lung_pid_gui.py`, `lung_hardware_monitor.py`, and `firmware/lung_controller_uno/lung_controller_uno.ino`, 2026.

[4] LungLogic project scope document, 2026.